

République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Systeme Intelligent de Gestion de Bibliothèque (Library Management System)

Conception et développement d'un backend REST API avec
Java, Spring Boot, JPA/Hibernate et MySQL

Équipe de développement :

- **24068** - Architecture, Sécurité, Gestion des exceptions (Leader)
- **24139** - Tables de référence (Language, Nationalité, Catégorie, Éditeur, Auteur)
- **24238** - Livres et exemplaires (Book, BookItem, BookAuthor)
- **24014** - Membres et emprunts (Member, Borrow)
- **24157** - Réservations et file d'attente FIFO (Reservation)

Encadrant :

Mr. [Nom de l'encadrant]

Année universitaire : 2025-2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Nous remercions vivement notre encadrant, **Mr. [Nom]**, pour sa disponibilité, ses conseils éclairés et son accompagnement tout au long de ce travail.

Nous remercions également l'administration de l'établissement pour nous avoir fourni les ressources nécessaires au bon déroulement de ce projet.

Enfin, nous exprimons notre reconnaissance envers nos familles et nos proches pour leur soutien indéfectible durant cette période.

Table des matières

Remerciements	2
Table des figures	6
Liste des tableaux	7
Glossaire et Abréviations	8
I Introduction	9
I.1 Contexte du projet	9
I.2 Objectifs	9
I.3 Équipe et répartition des tâches	9
2 Analyse et Conception	II
2.1 Cahier des charges	II
2.1.1 Besoins fonctionnels	II
2.1.2 Besoins non fonctionnels	II
2.2 Modèle Conceptuel de Données (MCD)	12
2.2.1 Diagramme entité-association	12
2.2.2 Liste des entités	12
2.3 Diagramme de cas d'utilisation	13
2.4 Diagramme de séquence - Emprunt d'un livre	13
2.5 Règles métier	14
3 Architecture Technique	15
3.1 Stack technologique	15
3.2 Structure du projet	15
3.2.1 Structure des packages	16
3.3 Spring Security	17
3.4 Gestion des exceptions	18
3.5 Configuration de la base de données	18
3.5.1 Profil de production (MySQL)	18
3.5.2 Profil de test (H2)	18

4	Implémentation par Membre	20
4.1	Membre 24068 – Architecture, Sécurité et Soft Delete	20
4.1.1	BaseEntity – Soft Delete	20
4.1.2	Entités avec Soft Delete	20
4.1.3	GlobalExceptionHandler	21
4.1.4	Fichiers créés par le membre 24068	22
4.2	Membre 24139 – Tables de référence	22
4.2.1	Entités créées	22
4.2.2	Services et Controllers	24
4.2.3	Fichiers créés par le membre 24139	26
4.3	Membre 24238 – Livres et Exemplaires	26
4.3.1	Entité Book	26
4.3.2	Entité BookItem – Optimistic Locking	27
4.3.3	Entité BookAuthor – Clé composée	28
4.3.4	Fichiers créés par le membre 24238	29
4.4	Membre 24014 – Membres et Emprunts	29
4.4.1	Entité Member	29
4.4.2	Entité Borrow	30
4.4.3	BorrowService – Règles métier	30
4.4.4	Fichiers créés par le membre 24014	33
4.5	Membre 24157 – Réservations et File d'attente FIFO	33
4.5.1	Entité Reservation	33
4.5.2	ReservationService – Logique FIFO	34
4.5.3	Repository avec requêtes FIFO	35
4.5.4	Endpoints REST	35
4.5.5	Fichiers créés par le membre 24157	36
5	API REST	37
5.1	Endpoints disponibles	37
5.2	Schémas de requêtes et réponses	38
5.2.1	Création d'un auteur	38
5.2.2	Emprunt d'un livre	38
5.2.3	Réservation avec file d'attente	38
5.3	Sécurisation des endpoints	39
6	Tests et Validation	40
6.1	Stratégie de test	40
6.1.1	Tests unitaires avec H2	40
6.1.2	Script de test API	40
6.2	Résultat du build	41

7	Workflow Git	42
7.1	Stratégie de branches	42
7.1.1	Branches utilisées	42
7.2	Historique des Pull Requests	43
8	Conclusion	44
8.1	Bilan du projet	44
8.2	Améliorations apportées	44
8.3	Perspectives	44
	Références bibliographiques	46
A	Annexes	47
A.1	Schéma SQL complet	47
A.2	Structure complète du projet	49
A.3	Commandes utiles	50

Table des figures

2.1	Modèle Conceptuel de Données du système	12
2.2	Diagramme de cas d'utilisation	13
2.3	Diagramme de séquence - Emprunt d'un livre	14
3.1	Architecture en couches du projet	16
7.1	Stratégie de branches	42

Liste des tableaux

1.1	Répartition des tâches par membre	10
3.1	Technologies utilisées	15
4.1	Travail du membre 24068	22
4.2	Travail du membre 24139	26
4.3	Travail du membre 24238	29
4.4	Travail du membre 24014	33
4.5	Travail du membre 24157	36
5.2	Authentification Basic Auth	39
7.1	Branches du projet	42

Glossaire et Abréviations

Abréviation	Signification
API	Application Programming Interface (Interface de Programmation Applicative)
CRUD	Create, Read, Update, Delete (Créer, Lire, Mettre à jour, Supprimer)
DDL	Data Definition Language (Langage de Définition des Données)
DTO	Data Transfer Object (Objet de Transfert de Données)
FIFO	First In, First Out (Premier Entré, Premier Sorti)
FK	Foreign Key (Clé Étrangère)
JPA	Jakarta Persistence API (API de Persistance Jakarta)
JSON	JavaScript Object Notation
JWT	JSON Web Token
MCD	Modèle Conceptuel de Données
ORM	Object-Relational Mapping (Mapping Objet-Relationnel)
PK	Primary Key (Clé Primaire)
PR	Pull Request (Demande d'Intégration)
REST	Representational State Transfer
SQL	Structured Query Language
UML	Unified Modeling Language (Langage de Modélisation Unifié)

Chapitre I

Introduction

I.1 Contexte du projet

La gestion manuelle des bibliothèques présente de nombreuses difficultés : suivi complexe des documents, risque d'erreurs humaines dans l'enregistrement des prêts, difficulté à gérer les files d'attente pour les ouvrages très demandés, et absence de traçabilité des opérations. Face à ces constats, ce projet vise à développer un **système intelligent de gestion de bibliothèque** permettant d'automatiser et de fluidifier l'ensemble des processus métier.

Ce système se veut une plateforme moderne, fiable et scalable, capable de gérer de manière autonome le catalogue d'œuvres, les exemplaires physiques, les adhérents et les mouvements de prêts et de réservations.

I.2 Objectifs

Les objectifs principaux de ce projet sont :

- Concevoir et développer une API REST complète pour la gestion d'une bibliothèque
- Implémenter des règles métier précises (quota d'emprunts, file d'attente FIFO, limite de renouvellements)
- Assurer l'intégrité des données via le Soft Delete et l'Optimistic Locking
- Sécuriser l'API avec Spring Security (Basic Auth)
- Travailler en équipe avec un workflow Git structuré (branches, Pull Requests)

I.3 Équipe et répartition des tâches

Le projet a été réalisé par une équipe de 5 membres, chacun responsable d'un module spécifique :

TABLE I.1 – Répartition des tâches par membre

Membre	Rôle	Tâche
24068	Leader	Architecture, Spring Security, BaseEntity, GlobalExceptionHandler
24139	Développeur	Tables de référence (Language, Nationality, Category, Publisher, Author)
24238	Développeur	Livres et exemplaires (Book, BookItem, BookAuthor)
24014	Développeur	Membres et emprunts (Member, Borrow, règles métier)
24157	Développeur	Réservations et file d'attente FIFO (Reservation)

Chapitre 2

Analyse et Conception

2.1 Cahier des charges

Le cahier des charges fonctionnel définit les besoins suivants :

2.1.1 Besoins fonctionnels

1. **Gestion du catalogue** : Ajouter, consulter et supprimer des livres, auteurs, catégories, éditeurs et langues
2. **Gestion des membres** : Enregistrer les membres avec leur type (Étudiant, Enseignant, Externe) et leur quota d'emprunts
3. **Gestion des emprunts** : Permettre l'emprunt, le retour et le renouvellement des exemplaires physiques
4. **Gestion des réservations** : Permettre aux membres de réserver un livre indisponible avec une file d'attente FIFO
5. **Sécurité** : Authentification par Basic Auth avec rôles ADMIN et USER
6. **Archivage** : Suppression logique (Soft Delete) pour préserver l'historique

2.1.2 Besoins non fonctionnels

- Architecture en couches (Controller, Service, Repository, Entity)
- Base de données relationnelle MySQL
- API RESTful avec endpoints cohérents
- Code maintenable et documenté
- Build automatisé avec Maven

2.2 Modèle Conceptuel de Données (MCD)

Le Modèle Conceptuel de Données constitue le socle de notre système. Il définit l'ensemble des entités et leurs relations.

2.2.1 Diagramme entité-association

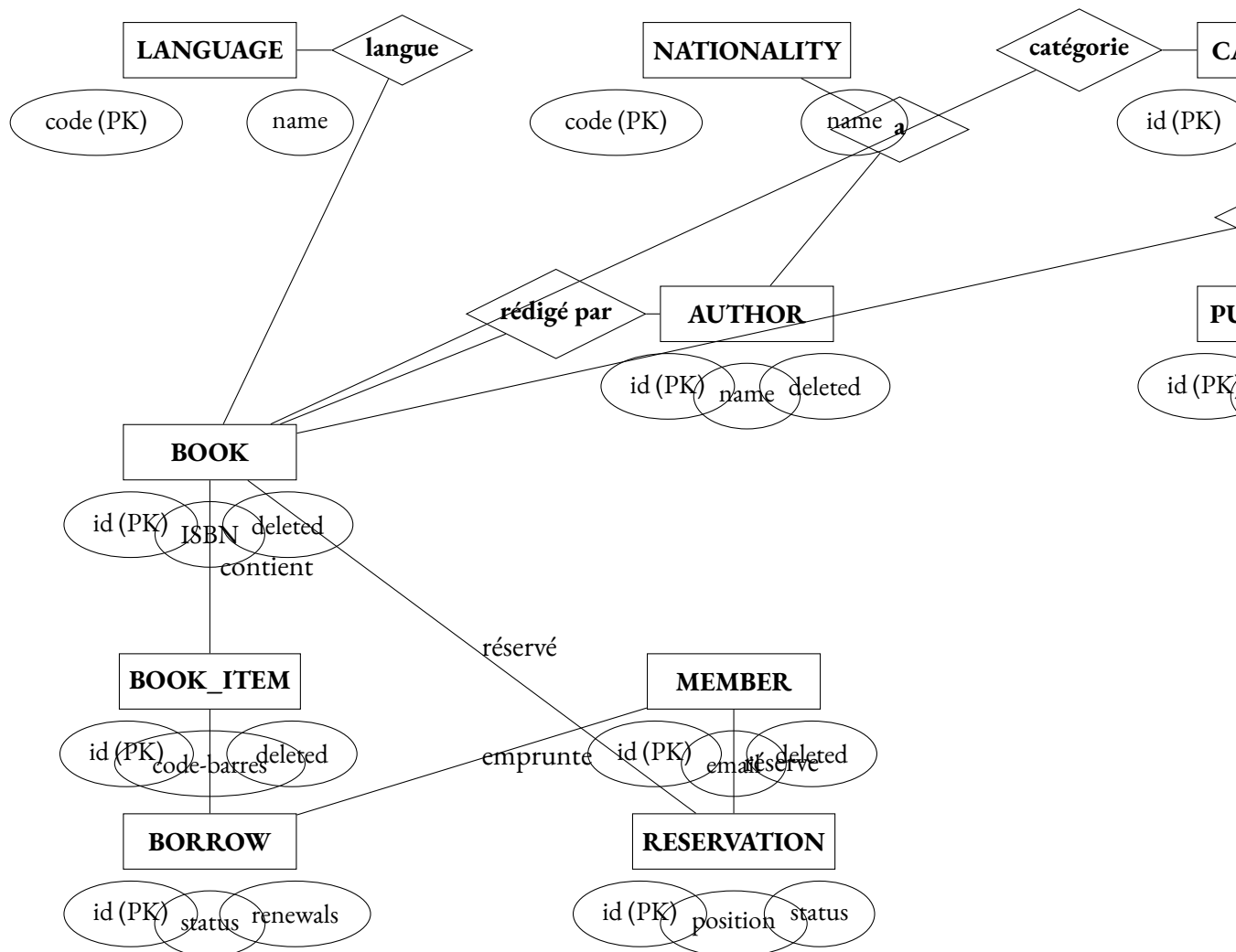


FIGURE 2.1 – Modèle Conceptuel de Données du système

2.2.2 Liste des entités

Le tableau suivant présente les 11 entités qui composent notre modèle de données :

Entité	Table	Description
LANGUAGE	language	Langue d'un livre (code ISO comme clé primaire)
NATIONALITY	nationality	Nationalité d'un auteur (code comme clé primaire)
CATEGORY	category	Catégorie thématique d'un livre (Soft Delete)

PUBLISHER	publisher	Maison d'édition (Soft Delete)
AUTHOR	author	Auteur avec nationalité (Soft Delete)
BOOK	book	Œuvre littéraire avec ISBN unique (Soft Delete)
BOOK_ITEM	book_item	Exemplaire physique avec code-barres (Soft Delete, Optimistic Locking)
BOOK_AUTHOR	book_author	Association livre-auteur with rôle (clé composée)
MEMBER	member	Membre de la bibliothèque (Soft Delete)
BORROW	borrow	Emprunt d'un exemplaire par un membre
RESERVATION	reservation	Réservation d'un livre avec file d'attente

2.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les interactions entre les différents acteurs et le système :

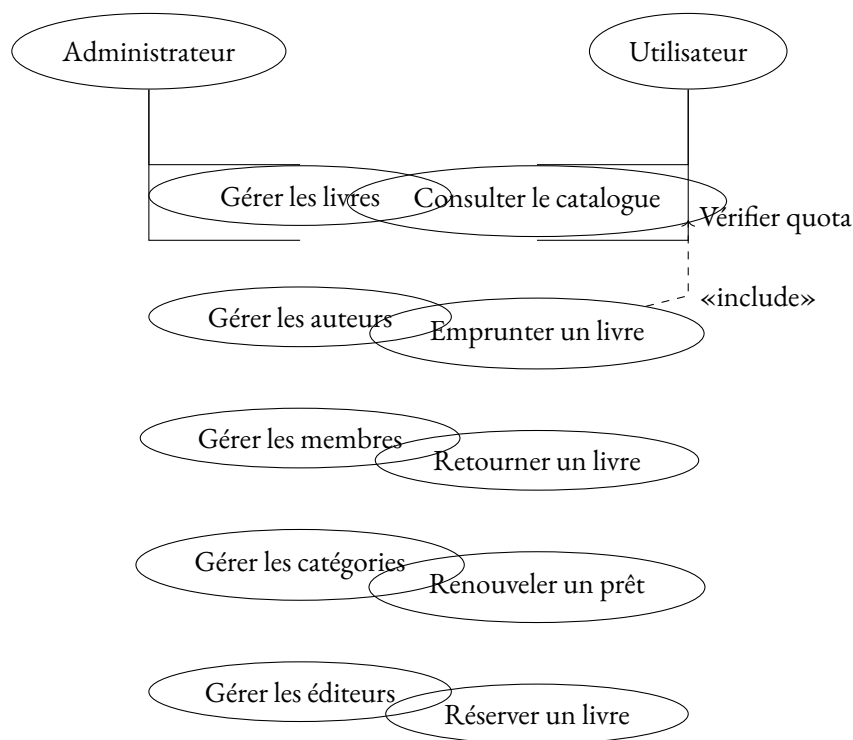


FIGURE 2.2 – Diagramme de cas d'utilisation

2.4 Diagramme de séquence - Emprunt d'un livre

La séquence ci-dessous illustre le flux complet d'un emprunt :

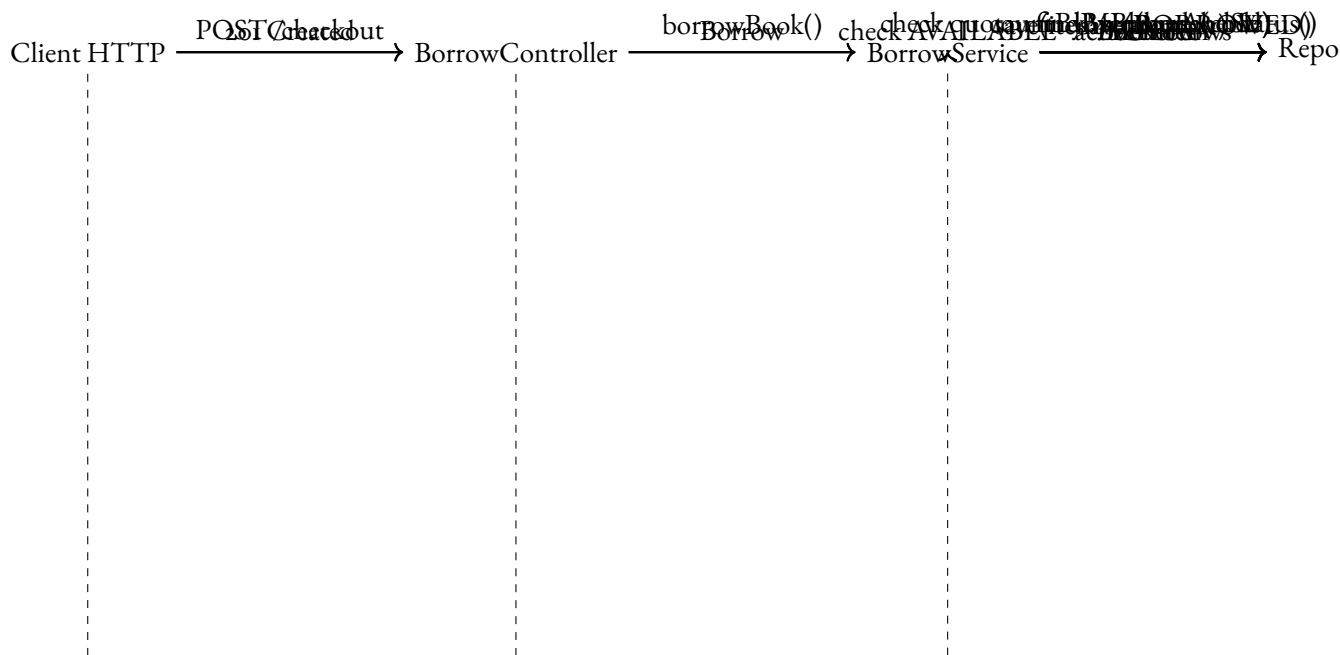


FIGURE 2.3 – Diagramme de séquence - Emprunt d'un livre

2.5 Règles métier

Le système implémente les règles métier suivantes :

Règle	Description	Implémentation
Séparation Livre/Exemplaire	Un catalogue d'œuvres, suivi individuel par code-barres	Entités distinctes Book et BookItem
Limite de prêt (Quota)	Quota par type de membre (maxBorrows)	Vérifié dans BorrowService.borrowBook()
Disponibilité	Un exemplaire doit être AVAILABLE pour être emprunté	Vérification du statut avant prêt
Renouvellement	Maximum 3 renouvellements par emprunt	Compteur renewalCount dans Borrow
File d'attente FIFO	Réservations traitées dans l'ordre d'arrivée	queuePosition calculée dans ReservationService
Soft Delete	Archivage sans perte des données historiques	BaseEntity + @SQLRestriction
Optimistic Locking	Empêche les modifications concurrentes sur BookItem	@Version sur le champ version

Chapitre 3

Architecture Technique

3.1 Stack technologique

TABLE 3.1 – Technologies utilisées

Technologie	Rôle
Java 21	Langage de programmation
Spring Boot 3.x	Framework de développement d'applications
Spring Data JPA	ORM et accès aux données
Spring Security	Sécurisation de l'API
Hibernate 6	Implémentation JPA
MySQL 8	Base de données relationnelle
Maven	Gestionnaire de build et de dépendances
Lombok	Réduction du code boilerplate
H2 Database	Base de données pour les tests
Jackson	Sérialisation JSON

3.2 Structure du projet

Le projet suit une architecture en couches (layered architecture) :

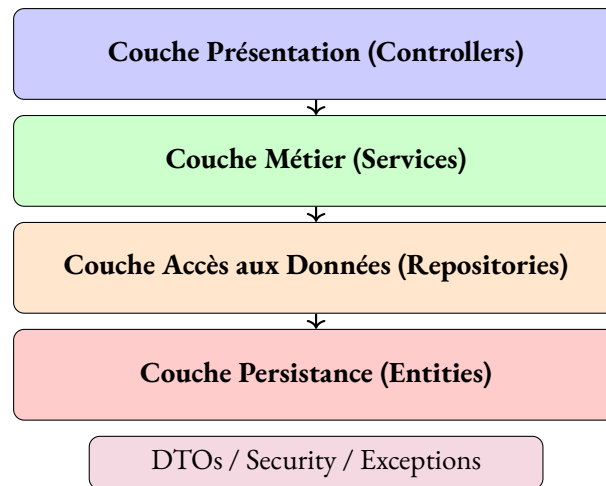


FIGURE 3.1 – Architecture en couches du projet

3.2.1 Structure des packages

```

1 src/main/java/supnum/projet/Library/
2   LibraryApplication.java
3   controllers/
4       AuthorController.java
5       BookController.java
6       BorrowController.java
7       CategoryController.java
8       PublisherController.java
9       ReservationController.java
10  data/
11    entities/
12      enums/
13          AuthorRole.java
14          BookItemStatus.java
15          BorrowStatus.java
16          MemberType.java
17          ReservationStatus.java
18      Author.java
19      BaseEntity.java
20      Book.java
21      BookAuthor.java
22      BookItem.java
23      Borrow.java
24      Category.java
25      Language.java
26      Member.java
27      Nationality.java
28      Publisher.java
29      Reservation.java
30  repositories/
  
```



```

31      AuthorRepository.java
32      BookItemRepository.java
33      BookRepository.java
34      BorrowRepository.java
35      CategoryRepository.java
36      LanguageRepository.java
37      MemberRepository.java
38      NationalityRepository.java
39      PublisherRepository.java
40      ReservationRepository.java
41  dto/
42      AuthorDTO.java
43      BookDTO.java
44      CategoryDTO.java
45      MemberDTO.java
46      PublisherDTO.java
47      ReservationDTO.java
48  exceptions/
49      GlobalExceptionHandler.java
50      ResourceNotFoundException.java
51  security/
52      SecurityConfig.java

```

Listing 3.1 – Organisation des packages Java

3.3 Spring Security

La configuration de sécurité est gérée par la classe `SecurityConfig.java`. Elle implémente :

- **Authentification HTTP Basic** avec deux utilisateurs prédéfinis
- **Autorisation par rôle** : les endpoints `/api/admin/**` nécessitent le rôle ADMIN, les autres endpoints sont accessibles aux utilisateurs authentifiés
- **Protection CSRF désactivée** pour une API REST
- **Chiffrement des mots de passe** avec BCrypt

```

1  @Configuration
2  @EnableWebSecurity
3  public class SecurityConfig {
4
5      @Bean
6      public SecurityFilterChain filterChain(HttpSecurity http) throws
       Exception {
7          http
8              .csrf(csrf -> csrf.disable())
9              .authorizeHttpRequests(auth -> auth

```

```
10         .requestMatchers("/api/public/**").permitAll()  
11         .requestMatchers("/api/admin/**").hasRole("ADMIN")  
12         .anyRequest().authenticated()  
13     )  
14     .httpBasic(basic -> {});  
15     return http.build();  
16 }  
17 // ...  
18 }
```

Listing 3.2 – Configuration de sécurité (extrait)

3.4 Gestion des exceptions

La gestion centralisée des exceptions est assurée par `GlobalExceptionHandler.java` :

- **ResourceNotFoundException** → HTTP 404 avec message d'erreur
- **MethodArgumentNotValidException** → HTTP 400 avec détails des champs invalides
- **Exception générique** → HTTP 500 avec message d'erreur

3.5 Configuration de la base de données

3.5.1 Profil de production (MySQL)

La configuration principale se trouve dans `application.properties` :

```
1 spring.datasource.url=jdbc:mysql://localhost:3306/library_db?  
    createDatabaseIfNotExist=true  
2 spring.datasource.username=supnum  
3 spring.datasource.password=Supnum  
4 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver  
5 spring.jpa.hibernate.ddl-auto=update  
6 spring.jpa.show-sql=true  
7 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.  
    MySQLDialect  
8 spring.sql.init.mode=never
```

Listing 3.3 – Configuration MySQL

3.5.2 Profil de test (H2)

Pour les tests unitaires, une base de données H2 en mémoire est configurée :

```
1 # application-test.properties
2 spring.datasource.url=jdbc:h2:mem:testdb
3 spring.datasource.driver-class-name=org.h2.Driver
4 spring.jpa.hibernate.ddl-auto=create-drop
5 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.
   H2Dialect
```

Listing 3.4 – Configuration H2 pour les tests

Chapitre 4

Implémentation par Membre

4.1 Membre 24068 – Architecture, Sécurité et Soft Delete

Le membre 24068 a joué le rôle de **leader technique** et a mis en place l'infrastructure du projet.

4.1.1 BaseEntity – Soft Delete

La classe BaseEntity constitue la classe mère de toutes les entités devant supporter la suppression logique. Elle contient un champ deleted qui, lorsqu'il est positionné à true, archive l'enregistrement sans le supprimer physiquement.

```
1 @MappedSuperclass
2 public abstract class BaseEntity {
3     @Column(name = "deleted", nullable = false)
4     private boolean deleted = false;
5     // ...
6 }
```

Listing 4.1 – BaseEntity avec Soft Delete

4.1.2 Entités avec Soft Delete

Les entités marquées avec l'annotation @SQLRestriction("deleted = false") sont automatiquement filtrées par Hibernate. Cela garantit que les enregistrements archivés n'apparaissent pas dans les requêtes quotidiennes tout en préservant l'historique :

- Category
- Publisher
- Author
- Book
- BookItem

— Member

Les entités `Language`, `Nationality`, `BookAuthor`, `Borrow` et `Reservation` ne nécessitent pas de `Soft Delete` car leurs données ne doivent pas être supprimées ou archivées.

4.1.3 GlobalExceptionHandler

Le gestionnaire global d'exceptions normalise les réponses HTTP pour toutes les erreurs :

```
1 @RestControllerAdvice
2 public class GlobalExceptionHandler {
3
4     @ExceptionHandler(ResourceNotFoundException.class)
5     public ResponseEntity<Map<String, String>> handleNotFound(
6         ResourceNotFoundException ex) {
7         // HTTP 404
8         return new ResponseEntity<>(Map.of("error", ex.getMessage()),
9             HttpStatus.NOT_FOUND);
10    }
11
12    @ExceptionHandler(MethodArgumentNotValidException.class)
13    public ResponseEntity<Map<String, String>> handleValidation(
14        MethodArgumentNotValidException ex) {
15        // HTTP 400 avec les erreurs de validation
16        Map<String, String> errors = new HashMap<>();
17        ex.getBindingResult().getFieldErrors()
18            .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
19        return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
20    }
21
22    @ExceptionHandler(Exception.class)
23    public ResponseEntity<Map<String, String>> handleGeneral(Exception
24        ex) {
25        // HTTP 500
26        return new ResponseEntity<>(Map.of("error", ex.getMessage()),
27            HttpStatus.INTERNAL_SERVER_ERROR);
28    }
29 }
```

Listing 4.2 – GlobalExceptionHandler (extrait)

4.1.4 Fichiers créés par le membre 24068

TABLE 4.1 – Travail du membre 24068

Fichier	Description
BaseEntity.java	Classe mère avec gestion du Soft Delete
SecurityConfig.java	Configuration Spring Security (Basic Auth)
ResourceNotFoundException.java	Exception personnalisée pour ressources introuvables
GlobalExceptionHandler.java	Gestionnaire centralisé des exceptions

4.2 Membre 24139 – Tables de référence

Le membre 24139 a implémenté l'ensemble des **tables de référence** qui permettent de structurer le catalogue de la bibliothèque.

4.2.1 Entités créées

Language Langue d'un livre (clé primaire : code ISO) :

```

1 @Entity
2 @Table(name = "language")
3 public class Language {
4     @Id
5     @Column(length = 10, nullable = false)
6     private String code;
7
8     @Column(length = 50, nullable = false)
9     private String name;
10    // ...
11 }
```

Listing 4.3 – Entité Language

Nationality Nationalité d'un auteur :

```

1 @Entity
2 @Table(name = "nationality")
3 public class Nationality {
4     @Id
5     @Column(length = 10, nullable = false)
6     private String code;
7
8     @Column(length = 50, nullable = false)
```

```
9     private String name;  
10    // ...  
11 }
```

Listing 4.4 – Entité Nationality

Category Catégorie thématique d'un livre (avec Soft Delete) :

```
1 @Entity  
2 @Table(name = "category")  
3 @SQLRestriction("deleted = false")  
4 public class Category extends BaseEntity {  
5     @Id  
6     @GeneratedValue(strategy = GenerationType.IDENTITY)  
7     private Long id;  
8  
9     @Column(nullable = false, unique = true, length = 100)  
10    private String name;  
11    // ...  
12 }
```

Listing 4.5 – Entité Category

Publisher Maison d'édition (avec Soft Delete et email) :

```
1 @Entity  
2 @Table(name = "publisher")  
3 @SQLRestriction("deleted = false")  
4 public class Publisher extends BaseEntity {  
5     @Id  
6     @GeneratedValue(strategy = GenerationType.IDENTITY)  
7     private Long id;  
8  
9     @Column(nullable = false, unique = true, length = 100)  
10    private String name;  
11  
12    @Column(unique = true, length = 100)  
13    private String email;  
14    // ...  
15 }
```

Listing 4.6 – Entité Publisher

Author Auteur avec nationalité (Soft Delete) :

```
1 @Entity  
2 @Table(name = "author")
```

```
3 @SQLRestriction("deleted = false")
4 public class Author extends BaseEntity {
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @Column(nullable = false, length = 100)
10    private String name;
11
12    @ManyToOne(fetch = FetchType.LAZY)
13    @JoinColumn(name = "nationality_code")
14    private Nationality nationality;
15    // ...
16 }
```

Listing 4.7 – Entité Author

4.2.2 Services et Controllers

Le service AuthorService illustre la logique métier pour la gestion des auteurs :

```
1 @Service
2 @Transactional
3 public class AuthorService {
4     private final AuthorRepository authorRepository;
5     private final NationalityRepository nationalityRepository;
6
7     public List<Author> findAll() {
8         return authorRepository.findAll();
9     }
10
11    public Author create(AuthorDTO dto) {
12        Nationality nationality = nationalityRepository.findById(dto.
13        getNationalityCode())
14        .orElseThrow(() -> new ResourceNotFoundException(
15        "Nationalite non trouvee avec le code : " + dto.
16        getNationalityCode()));
17        Author author = new Author();
18        author.setName(dto.getName());
19        author.setNationality(nationality);
20        return authorRepository.save(author);
21    }
22
23    public void delete(Long id) {
24        Author author = authorRepository.findById(id)
25        .orElseThrow(() -> new ResourceNotFoundException(
26        "Auteur non trouve avec l'id : " + id));
```



```
25     author.setDeleted(true);
26     authorRepository.save(author);
27 }
28 }
```

Listing 4.8 – AuthorService (extrait)

Le AuthorController expose les endpoints REST :

```
1 @RestController
2 @RequestMapping("/api/authors")
3 public class AuthorController {
4     private final AuthorService service;
5
6     public AuthorController(AuthorService service) {
7         this.service = service;
8     }
9
10    @GetMapping
11    public List<Author> getAll() {
12        return service.findAll();
13    }
14
15    @PostMapping
16    public ResponseEntity<Author> create(@Valid @RequestBody AuthorDTO
17    dto) {
18        return ResponseEntity.ok(service.create(dto));
19    }
20
21    @DeleteMapping("/{id}")
22    public ResponseEntity<Void> delete(@PathVariable Long id) {
23        service.delete(id);
24        return ResponseEntity.noContent().build();
25    }
26 }
```

Listing 4.9 – AuthorController

4.2.3 Fichiers créés par le membre 24139

TABLE 4.2 – Travail du membre 24139

Fichier	Description
Language.java	Entité langue
Nationality.java	Entité nationalité
Category.java	Entité catégorie
Publisher.java	Entité éditeur
Author.java	Entité auteur
LanguageRepository.java	Repository Language
NationalityRepository.java	Repository Nationality
CategoryRepository.java	Repository Category
PublisherRepository.java	Repository Publisher
AuthorRepository.java	Repository Author
CategoryDTO.java	DTO Category
AuthorDTO.java	DTO Author
PublisherDTO.java	DTO Publisher
CategoryService.java	Service Category
AuthorService.java	Service Author
PublisherService.java	Service Publisher
CategoryController.java	Controller Category
AuthorController.java	Controller Author
PublisherController.java	Controller Publisher

4.3 Membre 24238 – Livres et Exemplaires

Le membre 24238 a implémenté la gestion des **livres** (œuvre intellectuelle) et des **exemplaires** (objets physiques), avec le **verrouillage optimiste** et la **table d'association livre-auteur**.

4.3.1 Entité Book

L'entité Book représente l'œuvre littéraire avec son ISBN unique et ses relations vers la langue, la catégorie et l'éditeur :

```

1 @Entity
2 @Table(name = "book")
3 @SQLRestriction("deleted = false")
4 public class Book extends BaseEntity {
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)

```

```

7     private Long id;
8
9     @Column(nullable = false, length = 200)
10    private String title;
11
12    @Column(nullable = false, unique = true, length = 20)
13    private String isbn;
14
15    @ManyToOne(fetch = FetchType.LAZY)
16    @JoinColumn(name = "language_code")
17    private Language language;
18
19    @ManyToOne(fetch = FetchType.LAZY)
20    @JoinColumn(name = "category_id")
21    private Category category;
22
23    @ManyToOne(fetch = FetchType.LAZY)
24    @JoinColumn(name = "publisher_id")
25    private Publisher publisher;
26
27    @OneToMany(mappedBy = "book", cascade = CascadeType.ALL,
28    orphanRemoval = true)
29    private List<BookItem> bookItems;
30    // ...
31 }

```

Listing 4.10 – Entité Book

4.3.2 Entité BookItem – Optimistic Locking

L'entité BookItem représente l'exemplaire physique. Elle intègre le **verrouillage optimiste (Optimistic Locking)** via l'annotation @Version :

```

1 @Entity
2 @Table(name = "book_item")
3 @SQLRestriction("deleted = false")
4 public class BookItem extends BaseEntity {
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @Column(nullable = false, unique = true, length = 50)
10    private String barcode;
11
12    @Enumerated(EnumType.STRING)
13    @Column(nullable = false)
14    private BookItemStatus status = BookItemStatus.AVAILABLE;

```

```

15
16     @Version
17     private Long version; // Optimistic locking
18
19     @JsonIgnore
20     @ManyToOne(fetch = FetchType.LAZY)
21     @JoinColumn(name = "book_id")
22     private Book book;
23     // ...
24 }

```

Listing 4.11 – BookItem avec Optimistic Locking et JsonIgnore

L'annotation `@JsonIgnore` empêche la sérialisation récursive de la relation inverse lors de la génération du JSON, évitant ainsi les erreurs de `StackOverflow`.

4.3.3 Entité BookAuthor – Clé composée

La table d'association `BookAuthor` utilise une **clé primaire composée** pour lier un auteur à un livre avec un rôle spécifique :

```

1 @Entity
2 @Table(name = "book_author")
3 public class BookAuthor {
4     @EmbeddedId
5     private BookAuthorId id = new BookAuthorId();
6
7     @ManyToOne(fetch = FetchType.LAZY)
8     @MapsId("bookId")
9     @JoinColumn(name = "book_id")
10    private Book book;
11
12    @ManyToOne(fetch = FetchType.LAZY)
13    @MapsId("authorId")
14    @JoinColumn(name = "author_id")
15    private Author author;
16
17    @Enumerated(EnumType.STRING)
18    @Column(nullable = false)
19    private AuthorRole role;
20
21    @Embeddable
22    public static class BookAuthorId implements Serializable {
23        private Long bookId;
24        private Long authorId;
25        // equals() et hashCode() implementes
26    }
27 }

```

Listing 4.12 – BookAuthor avec clé composée

4.3.4 Fichiers créés par le membre 24238

TABLE 4.3 – Travail du membre 24238

Fichier	Description
BookItemStatus.java	Énumération des statuts (AVAILABLE, BORROWED, RESERVED, LOST, DAMAGED)
AuthorRole.java	Énumération des rôles (MAIN_AUTHOR, CO_AUTHOR, EDITOR)
Book.java	Entité livre
BookItem.java	Entité exemplaire avec @Version
BookAuthor.java	Entité d'association avec clé composée
BookRepository.java	Repository Book
BookItemRepository.java	Repository BookItem
BookDTO.java	DTO Book
BookService.java	Service Book
BookController.java	Controller Book

4.4 Membre 24014 – Membres et Emprunts

Le membre 24014 a implémenté la gestion des **membres** et des **emprunts**, incluant les règles métier de quota et de renouvellement.

4.4.1 Entité Member

```

1 @Entity
2 @Table(name = "member")
3 @SQLRestriction("deleted = false")
4 public class Member extends BaseEntity {
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @Column(nullable = false, unique = true, length = 100)
10    private String email;
11
12    @Enumerated(EnumType.STRING)
13    @Column(name = "member_type", nullable = false)

```

```
14     private MemberType memberType;
15
16     @Column(name = "max_borrows", nullable = false)
17     private Integer maxBorrows;
18     // ...
19 }
```

Listing 4.13 – Entité Member

4.4.2 Entité Borrow

```
1 @Entity
2 @Table(name = "borrow")
3 public class Borrow {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @ManyToOne(fetch = FetchType.LAZY)
9     @JoinColumn(name = "member_id", nullable = false)
10    private Member member;
11
12    @ManyToOne(fetch = FetchType.LAZY)
13    @JoinColumn(name = "book_item_id", nullable = false)
14    private BookItem bookItem;
15
16    @Column(name = "renewal_count", nullable = false)
17    private Integer renewalCount = 0;
18
19    @Enumerated(EnumType.STRING)
20    @Column(nullable = false)
21    private BorrowStatus status = BorrowStatus.ACTIVE;
22    // ...
23 }
```

Listing 4.14 – Entité Borrow

4.4.3 BorrowService – Règles métier

Le service BorrowService implémente trois opérations fondamentales :

Emprunt d'un livre

La méthode borrowBook() vérifie deux conditions avant d'autoriser l'emprunt :

1. **Disponibilité de l'exemplaire** : Le BookItem doit être en statut AVAILABLE

2. Quota du membre : Le nombre d'emprunts actifs ne doit pas dépasser maxBorrows

```
1 public Borrow borrowBook(Long memberId, String barcode) {
2     Member member = memberRepository.findById(memberId)
3         .orElseThrow(() -> new ResourceNotFoundException("Membre
4         introuvable"));
5     BookItem item = bookItemRepository.findByBarcode(barcode)
6         .orElseThrow(() -> new ResourceNotFoundException("Exemplaire
7         introuvable"));
8
9     if (item.getStatus() != BookItemStatus.AVAILABLE) {
10         throw new RuntimeException("L'exemplaire n'est pas disponible
11         ");
12     }
13
14     long activeBorrows = borrowRepository.countByMemberAndStatus(
15         member, BorrowStatus.ACTIVE);
16     if (activeBorrows >= member.getMaxBorrows()) {
17         throw new RuntimeException("Le membre a atteint sa limite
18         maximale d'emprunts");
19     }
20
21     item.setStatus(BookItemStatus.BORROWED);
22     bookItemRepository.save(item);
23
24     Borrow borrow = new Borrow();
25     borrow.setMember(member);
26     borrow.setBookItem(item);
27     borrow.setStatus(BorrowStatus.ACTIVE);
28     return borrowRepository.save(borrow);
29 }
```

Listing 4.15 – Méthode borrowBook (extrait)

Retour d'un livre

```
1 public Borrow returnBook(Long borrowId) {
2     Borrow borrow = borrowRepository.findById(borrowId)
3         .orElseThrow(() -> new ResourceNotFoundException("Emprunt
4         introuvable"));
5
6     if (borrow.getStatus() != BorrowStatus.ACTIVE) {
7         throw new RuntimeException("Cet emprunt n'est plus actif");
8     }
9
10    borrow.setStatus(BorrowStatus.RETURNED);
11    BookItem item = borrow.getBookItem();
```

```
11     item.setStatus(BookItemStatus.AVAILABLE);
12     bookItemRepository.save(item);
13     return borrowRepository.save(borrow);
14 }
```

Listing 4.16 – Méthode returnBook (extrait)

Renouvellement d'un prêt

La règle stipule qu'un emprunt ne peut être renouvelé que **3 fois maximum** :

```
1 public Borrow renewBorrow(Long borrowId) {
2     Borrow borrow = borrowRepository.findById(borrowId)
3         .orElseThrow(() -> new ResourceNotFoundException("Emprunt
4             introuvable"));
5
6     if (borrow.getStatus() != BorrowStatus.ACTIVE) {
7         throw new RuntimeException("Seuls les emprunts actifs peuvent
8             etre renouveles");
9     }
10
11     if (borrow.getRenewalCount() >= 3) {
12         throw new RuntimeException("Limite de renouvellement atteinte
13             (max 3 fois)");
14     }
15
16     borrow.setRenewalCount(borrow.getRenewalCount() + 1);
17     return borrowRepository.save(borrow);
18 }
```

Listing 4.17 – Méthode renewBorrow (extrait)

4.4.4 Fichiers créés par le membre 24014

TABLE 4.4 – Travail du membre 24014

Fichier	Description
MemberType.java	Énumération des types (STUDENT, TEACHER, EXTERNAL)
BorrowStatus.java	Énumération des statuts (ACTIVE, RETURNED, OVERDUE, LOST)
Member.java	Entité membre
Borrow.java	Entité emprunt
MemberRepository.java	Repository Member
BorrowRepository.java	Repository Borrow
MemberDTO.java	DTO Member
BorrowService.java	Service avec règles métier
BorrowController.java	Controller Borrow

4.5 Membre 24157 – Réservations et File d'attente FIFO

Le membre 24157 a implémenté le système de **réservations** avec une **file d'attente FIFO** (First In, First Out).

4.5.1 Entité Reservation

```

1 @Entity
2 @Table(name = "reservation")
3 public class Reservation {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private Long id;
7
8     @ManyToOne(fetch = FetchType.LAZY)
9     @JoinColumn(name = "member_id", nullable = false)
10    private Member member;
11
12    @ManyToOne(fetch = FetchType.LAZY)
13    @JoinColumn(name = "book_id", nullable = false)
14    private Book book;
15
16    @Column(name = "queue_position", nullable = false)
17    private Integer queuePosition;
18

```

```
19 @Enumerated(EnumType.STRING)
20 @Column(nullable = false)
21 private ReservationStatus status = ReservationStatus.PENDING;
22 // ...
23 }
```

Listing 4.18 – Entité Reservation

4.5.2 ReservationService – Logique FIFO

```
1 @Service
2 @Transactional
3 public class ReservationService {
4
5     public Reservation reserve(ReservationDTO dto) {
6         Member member = memberRepository.findById(dto.getMemberId())
7             .orElseThrow(() -> new ResourceNotFoundException("Membre
non trouve"));
8         Book book = bookRepository.findById(dto.getBookId())
9             .orElseThrow(() -> new ResourceNotFoundException("Livre
non trouve"));
10
11         // Calcul automatique de la position FIFO
12         int nextPosition = reservationRepository.
findMaxQueuePositionForBook(book) + 1;
13
14         Reservation reservation = new Reservation();
15         reservation.setMember(member);
16         reservation.setBook(book);
17         reservation.setStatus(ReservationStatus.PENDING);
18         reservation.setQueuePosition(nextPosition);
19         return reservationRepository.save(reservation);
20     }
21
22     public Reservation cancel(Long reservationId) {
23         Reservation reservation = reservationRepository.findById(
reservationId)
24             .orElseThrow(() -> new ResourceNotFoundException("
Reservation non trouvee"));
25
26         if (reservation.getStatus() != ReservationStatus.PENDING) {
27             throw new RuntimeException("Seules les reservations en
attente peuvent etre annulees");
28         }
29
30         reservation.setStatus(ReservationStatus.CANCELLED);
}
```

```

31     return reservationRepository.save(reservation);
32 }
33
34 public List<Reservation> getQueueForBook(Long bookId) {
35     Book book = bookRepository.findById(bookId)
36         .orElseThrow(() -> new ResourceNotFoundException("Livre
non trouve"));
37     return reservationRepository
38         .findByBookAndStatusOrderByQueuePositionAsc(book,
ReservationStatus.PENDING);
39 }
40 }

```

Listing 4.19 – ReservationService avec file d’attente FIFO

4.5.3 Repository avec requêtes FIFO

Le repository utilise une requête JPQL pour calculer la position maximale dans la file :

```

1 @Repository
2 public interface ReservationRepository extends JpaRepository<
Reservation, Long> {
3
4     List<Reservation> findByBookAndStatusOrderByQueuePositionAsc(Book
book,
5         ReservationStatus status);
6
7     @Query("SELECT COALESCE(MAX(r.queuePosition), 0) FROM Reservation
r "
8         + "WHERE r.book = :book AND r.status = 'PENDING'")
9     Integer findMaxQueuePositionForBook(@Param("book") Book book);
10 }

```

Listing 4.20 – ReservationRepository

4.5.4 Endpoints REST

```

1 @RestController
2 @RequestMapping("/api/reservations")
3 public class ReservationController {
4
5     @PostMapping
6     public ResponseEntity<Reservation> reserve(@Valid @RequestBody
ReservationDTO dto) {
7         return ResponseEntity.ok(service.reserve(dto));
8     }
9 }

```

```

9
10     @PostMapping("/{id}/cancel")
11     public ResponseEntity<Reservation> cancel(@PathVariable Long id) {
12         return ResponseEntity.ok(service.cancel(id));
13     }
14
15     @GetMapping("/queue/{bookId}")
16     public ResponseEntity<List<Reservation>> getQueue(@PathVariable
17     Long bookId) {
18         return ResponseEntity.ok(service.getQueueForBook(bookId));
19     }

```

Listing 4.21 – ReservationController

4.5.5 Fichiers créés par le membre 24157

TABLE 4.5 – Travail du membre 24157

Fichier	Description
ReservationStatus.java	Énumération des statuts (PENDING, READY, CANCELLED, COMPLETED)
Reservation.java	Entité réservation avec file d'attente
ReservationRepository.java	Repository avec requêtes FIFO
ReservationDTO.java	DTO Reservation
ReservationService.java	Service avec logique FIFO
ReservationController.java	Controller Reservation

Chapitre 5

API REST

5.1 Endpoints disponibles

L'API expose les endpoints suivants :

Méthode	Endpoint	Contrôleur	Description
GET	/api/categories	CategoryController	Liste toutes les catégories
POST	/api/categories	CategoryController	Crée une catégorie
DELETE	/api/categories/{id}	CategoryController	Supprime (archive) une catégorie
GET	/api/authors	AuthorController	Liste tous les auteurs
POST	/api/authors	AuthorController	Crée un auteur
DELETE	/api/authors/{id}	AuthorController	Supprime (archive) un auteur
GET	/api/publishers	PublisherController	Liste tous les éditeurs
POST	/api/publishers	PublisherController	Crée un éditeur
DELETE	/api/publishers/{id}	PublisherController	Supprime (archive) un éditeur
GET	/api/books	BookController	Liste tous les livres
POST	/api/books	BookController	Crée un livre
POST	/api/borrows/checkout	BorrowController	Emprunte un livre
POST	/api/borrows/{id}/return	BorrowController	Retourne un livre
POST	/api/borrows/{id}/renew	BorrowController	Renouvelle un emprunt
POST	/api/reservations	ReservationController	Réserve un livre
POST	/api/reservations/{id}/cancel	ReservationController	Annule une réservation
GET	/api/reservations/queue/{bookId}	ReservationController	File d'attente d'un livre

5.2 Schémas de requêtes et réponses

5.2.1 Création d'un auteur

Requête POST /api/authors :

```
1 {  
2   "name": "Victor Hugo",  
3   "nationalityCode": "FR"  
4 }
```

Listing 5.1 – Requête POST /api/authors

Réponse (201 Created) :

```
1 {  
2   "id": 1,  
3   "name": "Victor Hugo",  
4   "nationality": {  
5     "code": "FR",  
6     "name": "France"  
7   }  
8 }
```

Listing 5.2 – Réponse POST /api/authors

5.2.2 Emprunt d'un livre

Requête POST /api/borrows/checkout?memberId=1&barcode=A001 :

```
1 {  
2   "id": 1,  
3   "member": { "id": 1, "email": "etudiant@example.com" },  
4   "bookItem": { "id": 1, "barcode": "A001", "status": "BORROWED" },  
5   "renewalCount": 0,  
6   "status": "ACTIVE"  
7 }
```

Listing 5.3 – Réponse POST /api/borrows/checkout

5.2.3 Réservation avec file d'attente

Requête POST /api/reservations :

```
1 {  
2   "memberId": 1,  
3   "bookId": 1  
4 }
```

Listing 5.4 – Requête POST /api/reservations

Réponse (position calculée automatiquement) :

```
1 {  
2   "id": 1,  
3   "member": { "id": 1, "email": "etudiant@example.com" },  
4   "book": { "id": 1, "title": "Les Miserables" },  
5   "queuePosition": 1,  
6   "status": "PENDING"  
7 }
```

Listing 5.5 – Réponse POST /api/reservations

5.3 Sécurisation des endpoints

La sécurité est configurée avec Spring Security et Basic Auth :

TABLE 5.2 – Authentication Basic Auth

Utilisateur	Mot de passe	Rôle
admin	admin123	ADMIN
user	user123	USER

Les utilisateurs non authentifiés ne peuvent accéder à aucun endpoint. Les utilisateurs avec le rôle ADMIN peuvent accéder aux endpoints /api/admin/**.

Chapitre 6

Tests et Validation

6.1 Stratégie de test

Deux approches de test ont été mises en œuvre :

6.1.1 Tests unitaires avec H2

Les tests unitaires utilisent une base de données H2 en mémoire, configurée via le profil Spring test. Cela permet de :

- Exécuter les tests sans dépendance à MySQL
- Isoler chaque test avec un rollback automatique
- Valider les règles métier (quota, FIFO, renouvellements)

6.1.2 Script de test API

Un script shell `test_api.sh` a été développé pour tester manuellement l'API :

```
1 #!/bin/bash
2 BASE_URL="http://localhost:8081/api"
3 AUTH="admin:admin123"
4
5 echo "=== Test Create Author ==="
6 curl -s -X POST "$BASE_URL/authors" \
7     -u "$AUTH" \
8     -H "Content-Type: application/json" \
9     -d '{"name": "Victor Hugo", "nationalityCode": "FR"}'
10 echo ""
11
12 echo "=== Test Create Book ==="
13 curl -s -X POST "$BASE_URL/books" \
14     -u "$AUTH" \
15     -H "Content-Type: application/json" \
```



```
16 -d '{"title":"Les Miserables","isbn":"978-2-07-040922-8",  
17     "languageCode":"FR","categoryId":1,"publisherId":1}'  
18 echo ""  
19  
20 echo "=== Test Borrow Book ==="  
21 curl -s -X POST "$BASE_URL/borrows/checkout?memberId=1&barcode=A001" -  
    u "$AUTH"  
22 echo ""
```

Listing 6.1 – Extrait du script de test API

6.2 Résultat du build

Le projet compile avec succès :

```
1 [INFO] Scanning for projects...  
2 [INFO] Building Library 0.0.1-SNAPSHOT  
3 [INFO] Compiling 49 source files with javac [debug parameters release  
    21]  
4 [INFO] BUILD SUCCESS  
5 [INFO] Total time: 4.575 s
```

Listing 6.2 – Build Maven réussi

Chapitre 7

Workflow Git

7.1 Stratégie de branches

Le projet a suivi une stratégie de branches structurée :

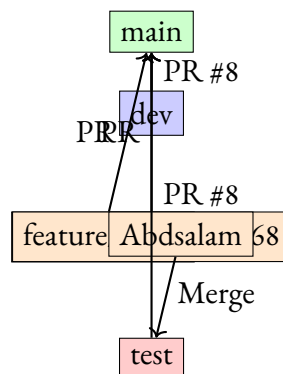


FIGURE 7.1 – Stratégie de branches

7.1.1 Branches utilisées

TABLE 7.1 – Branches du projet

Branch	Description	Membre
main	Branche principale de production	–
dev	Branche d'intégration	–
feature/24157	Développement réservations	24157
feature/taches-24068	Développement architecture	24068 (Leader)
Abdsalam	Développement tables de référence	24139
test	Branche de test	–

7.2 Historique des Pull Requests

1. **PR #1** : feature/taches-24068 → main (Structure initiale)
2. **PR #2** : feature/taches-24068 → main (Fix SecurityConfig)
3. **PR #3** : feature/taches-24068 → main (Configuration DB)
4. **PR #4** : feature/24157 → main (Réservations)
5. **PR #5** : feature/taches-24068 → main (Documentation)
6. **PR #6** : feature/taches-24068 → main (README final)
7. **PR #7** : feature/24157 → main (Finalisation réservations)
8. **PR #8** : dev → main (Intégration finale + améliorations)

Chapitre 8

Conclusion

8.1 Bilan du projet

Ce projet de système intelligent de gestion de bibliothèque a atteint l'ensemble de ses objectifs :

- **100% des fonctionnalités** demandées ont été implémentées
- **11 entités JPA** modélisant fidèlement le MCD
- **5 énumérations** pour les statuts et types
- **10 repositories** avec requêtes personnalisées
- **5 DTOs** avec validation Jakarta
- **6 services** avec règles métier
- **6 contrôleurs REST** exposant l'API
- **Sécurisation** de l'API avec Spring Security
- **49 fichiers Java** compilés avec succès

8.2 Améliorations apportées

Au-delà des spécifications initiales, les améliorations suivantes ont été intégrées :

- **@JsonIgnore** sur BookItem pour éviter les récursions JSON
- **Messages d'erreur détaillés** avec identification des ressources
- **Noms de variables explicites** dans les services
- **Script de test API** automatisé

8.3 Perspectives

Le système peut être étendu avec :

- Authentification JWT pour remplacer Basic Auth
- Interface frontend (React, Angular ou Vue.js)
- Notifications par email pour les réservations
- Statistiques et reporting des emprunts
- Gestion des amendes pour les retards
- Documentation Swagger/OpenAPI interactive

Références bibliographiques

1. Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
2. Spring Data JPA Reference Guide. <https://docs.spring.io/spring-data/jpa/reference/>
3. Spring Security Architecture. <https://docs.spring.io/spring-security/reference/>
4. Jakarta Persistence API 3.1 Specification. <https://jakarta.ee/specifications/persistence/3.1/>
5. Hibernate ORM Documentation. <https://hibernate.org/orm/documentation/6.4/>
6. UML 2.5 Specification. Object Management Group. <https://www.omg.org/spec/UML/>
7. Pro Git Book. Scott Chacon and Ben Straub. <https://git-scm.com/book/>

Chapitre A

Annexes

A.I Schéma SQL complet

```
1 CREATE TABLE language (  
2     code VARCHAR(10) PRIMARY KEY,  
3     name VARCHAR(50) NOT NULL  
4 );  
5  
6 CREATE TABLE nationality (  
7     code VARCHAR(10) PRIMARY KEY,  
8     name VARCHAR(50) NOT NULL  
9 );  
10  
11 CREATE TABLE category (  
12     id BIGINT AUTO_INCREMENT PRIMARY KEY,  
13     name VARCHAR(100) NOT NULL UNIQUE,  
14     deleted BOOLEAN NOT NULL DEFAULT FALSE  
15 );  
16  
17 CREATE TABLE publisher (  
18     id BIGINT AUTO_INCREMENT PRIMARY KEY,  
19     name VARCHAR(100) NOT NULL UNIQUE,  
20     email VARCHAR(100) UNIQUE,  
21     deleted BOOLEAN NOT NULL DEFAULT FALSE  
22 );  
23  
24 CREATE TABLE author (  
25     id BIGINT AUTO_INCREMENT PRIMARY KEY,  
26     name VARCHAR(100) NOT NULL,  
27     nationality_code VARCHAR(10),  
28     deleted BOOLEAN NOT NULL DEFAULT FALSE,  
29     FOREIGN KEY (nationality_code) REFERENCES nationality(code)  
30 );
```

```
31
32 CREATE TABLE book (
33     id BIGINT AUTO_INCREMENT PRIMARY KEY,
34     title VARCHAR(200) NOT NULL,
35     isbn VARCHAR(20) NOT NULL UNIQUE,
36     language_code VARCHAR(10),
37     category_id BIGINT,
38     publisher_id BIGINT,
39     deleted BOOLEAN NOT NULL DEFAULT FALSE,
40     FOREIGN KEY (language_code) REFERENCES language(code),
41     FOREIGN KEY (category_id) REFERENCES category(id),
42     FOREIGN KEY (publisher_id) REFERENCES publisher(id)
43 );
44
45 CREATE TABLE book_item (
46     id BIGINT AUTO_INCREMENT PRIMARY KEY,
47     barcode VARCHAR(50) NOT NULL UNIQUE,
48     status VARCHAR(20) NOT NULL DEFAULT 'AVAILABLE',
49     version BIGINT,
50     book_id BIGINT,
51     deleted BOOLEAN NOT NULL DEFAULT FALSE,
52     FOREIGN KEY (book_id) REFERENCES book(id)
53 );
54
55 CREATE TABLE book_author (
56     book_id BIGINT NOT NULL,
57     author_id BIGINT NOT NULL,
58     role VARCHAR(20) NOT NULL,
59     PRIMARY KEY (book_id, author_id),
60     FOREIGN KEY (book_id) REFERENCES book(id),
61     FOREIGN KEY (author_id) REFERENCES author(id)
62 );
63
64 CREATE TABLE member (
65     id BIGINT AUTO_INCREMENT PRIMARY KEY,
66     email VARCHAR(100) NOT NULL UNIQUE,
67     member_type VARCHAR(20) NOT NULL,
68     max_borrows INT NOT NULL,
69     deleted BOOLEAN NOT NULL DEFAULT FALSE
70 );
71
72 CREATE TABLE borrow (
73     id BIGINT AUTO_INCREMENT PRIMARY KEY,
74     member_id BIGINT NOT NULL,
75     book_item_id BIGINT NOT NULL,
76     renewal_count INT NOT NULL DEFAULT 0,
77     status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
```



```
78 FOREIGN KEY (member_id) REFERENCES member(id),
79 FOREIGN KEY (book_item_id) REFERENCES book_item(id)
80 );
81
82 CREATE TABLE reservation (
83     id BIGINT AUTO_INCREMENT PRIMARY KEY,
84     member_id BIGINT NOT NULL,
85     book_id BIGINT NOT NULL,
86     queue_position INT NOT NULL,
87     status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
88     FOREIGN KEY (member_id) REFERENCES member(id),
89     FOREIGN KEY (book_id) REFERENCES book(id)
90 );
```

Listing A.1 – Schéma de la base de données (DDL)

A.2 Structure complète du projet

```
1 Library/
2     pom.xml
3     README.md
4     description.md
5     24014.md
6     24068.md
7     24139.md
8     24157.md
9     24238.md
10    src/
11        main/
12            java/supnum/projet/Library/
13                LibraryApplication.java
14            controllers/
15                AuthorController.java
16                BookController.java
17                BorrowController.java
18                CategoryController.java
19                PublisherController.java
20                ReservationController.java
21            data/
22                entities/
23                    enums/
24                        AuthorRole.java
25                        BookItemStatus.java
26                        BorrowStatus.java
27                        MemberType.java
28                        ReservationStatus.java
```

```
29         Author.java
30         BaseEntity.java
31         Book.java
32         BookAuthor.java
33         BookItem.java
34         Borrow.java
35         Category.java
36         Language.java
37         Member.java
38         Nationality.java
39         Publisher.java
40         Reservation.java
41     repositories/
42         AuthorRepository.java
43         BookItemRepository.java
44         BookRepository.java
45         BorrowRepository.java
46         CategoryRepository.java
47         LanguageRepository.java
48         MemberRepository.java
49         NationalityRepository.java
50         PublisherRepository.java
51         ReservationRepository.java
52     dto/
53         AuthorDTO.java
54         BookDTO.java
55         CategoryDTO.java
56         MemberDTO.java
57         PublisherDTO.java
58         ReservationDTO.java
59     exceptions/
60         GlobalExceptionHandler.java
61         ResourceNotFoundException.java
62     security/
63         SecurityConfig.java
64     resources/
65         application.properties
66     test/java/.../
67         LibraryApplicationTests.java
68     test_api.sh
```

Listing A.2 – Arborescence complète du projet

A.3 Commandes utiles

```
1 # Lancer l'application
2 ./mvnw spring-boot:run
3
4 # Compiler sans tests
5 ./mvnw clean compile -DskipTests
6
7 # Ex cuter les tests
8 ./mvnw test
9
10 # Tester l'API avec curl
11 curl -s -X GET http://localhost:8081/api/authors -u admin:admin123
12 curl -s -X GET http://localhost:8081/api/books -u admin:admin123
13 curl -s -X POST "http://localhost:8081/api/borrows/checkout?memberId
    =1&barcode=A001" \
14     -u admin:admin123
15
16 # Utiliser le script de test automatisé
17 ./test_api.sh
```

Listing A.3 – Commandes pour lancer et tester l'application